

# Prompt Profiling Engine Engineering Report

Two-Stage XGBoost Architecture with Semantic Embeddings

June 2026

1. Executive Summary
2. Problem Statement
3. Framework Mapping: Scoring Dimensions
4. Dataset
5. Architecture
6. Implementation Details
7. Evaluation Results
8. Example Predictions
9. Explainability
10. Limitations
11. Conclusion

## 1. Executive Summary

The Prompt Profiling Engine classifies enterprise prompts into three complexity tiers (T1 Simple, T2 Medium, T3 Complex) by predicting five scoring dimensions defined in the Prompt Profiling specification. The system operates without LLM inference at classification time, using sentence embeddings and gradient-boosted classifiers to produce deterministic, explainable tier assignments.

**Key results (5-fold cross-validation, 889 samples):**

| Metric                            | Value                        |
|-----------------------------------|------------------------------|
| Tier classification accuracy      | 89.9% ± 1.3%                 |
| T1 (Simple) recall                | 93%                          |
| T2 (Medium) recall                | 86%                          |
| T3 (Complex) recall               | 90%                          |
| Dimension R² range                | 0.75 – 0.85                  |
| Inference latency (single prompt) | < 50ms (excluding embedding) |
| Model artifact size               | ~12 MB total                 |

All five dimension scores, the composite complexity score, and the tier assignment are returned per inference call, providing full transparency into the routing decision.

## 2. Problem Statement

Enterprise LLM routing systems need to assign inbound prompts to the appropriate model tier before the prompt reaches an LLM. This classification must be:

- Fast** — sub-100ms to avoid adding latency to the routing pipeline
- Deterministic** — the same prompt should always receive the same tier
- Explainable** — engineers must be able to inspect *why* a prompt was routed to a specific tier
- Non-LLM** — the classifier itself cannot call an LLM (circular dependency; cost prohibitive at high QPS)

The Prompt Profiling specification defines a five-dimension scoring rubric with configurable weights that produces a composite complexity score. This score maps to one of three tiers:

| Tier | Label   | Score Range | Model Pool                                      |
|------|---------|-------------|-------------------------------------------------|
| T1   | Simple  | 0.00 – 0.39 | Economy (e.g., Gemini Flash Lite, Nova Micro)   |
| T2   | Medium  | 0.40 – 0.69 | Balanced (e.g., GPT-4o-mini, Claude Haiku)      |
| T3   | Complex | 0.70 – 1.00 | Premium (e.g., GPT-4o, Claude Opus, Gemini Pro) |

The engineering challenge: **predict these five dimension scores from raw prompt text alone**, then apply the deterministic formula and tier thresholds.

## 3. Framework Mapping: Scoring Dimensions

The Prompt Profiling specification defines five dimensions, each scored on a discrete 5-point scale {0.0, 0.25, 0.50, 0.75, 1.0}. The composite score formula is:

ComplexityScore = (D1 × 0.35) + (D2 × 0.20) + (D3 × 0.20) + (D4 × 0.15) + (D5 × 0.10)

Below is the mapping from the specification’s rubric definitions to the implementation’s feature engineering and prediction strategy for each dimension.

### 3.1 D1 — Semantic Complexity (Weight: 0.35)

**Rubric definition:** Reasoning depth required — from factual recall (0.0) to cross-domain strategic synthesis requiring multi-agent orchestration (1.0).

| Score | Rubric Anchor                                                  | Implementation Signal                                                                                     |
|-------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 0.0   | Pure factual recall; single known answer                       | Short prompts, question marks, no action verbs                                                            |
| 0.25  | Simple comparison; 2-3 known entities                          | <code>has_comparison</code> feature fires; low word count                                                 |
| 0.50  | Multi-step analytical reasoning within one domain              | Moderate <code>action_verb_count</code> ; single domain keywords                                          |
| 0.75  | Cross-domain synthesis; structured argumentation               | <code>has_scope_words</code> ("strategic", "cross-domain", "synthesize"); multiple systems mentioned      |
| 1.0   | Strategic analysis; novel synthesis; multi-agent orchestration | <code>multi_stage_signal</code> + high <code>action_verb_count</code> + <code>stakeholder_mentions</code> |

**How the system predicts D1:** The model relies on a combination of semantic embedding features (PCA components capturing prompt meaning) and engineered structural features. The `has_scope_words` feature detects strategic-complexity language ("multi-stage", "cross-domain", "governance", "operating model"). The `action_verb_count` tallies high-agency verbs ("build", "design", "evaluate", "restructure", "optimize"), which correlate strongly with reasoning depth. The `multi_stage_signal` identifies phased workflows. D1 carries the highest weight (0.35) in the composite formula, making it the single most influential dimension for tier assignment.

**Achieved performance:**  $R^2 = 0.8455$ , MAE = 0.0582, Accuracy = 79.19%

### 3.2 D2 — Domain Specificity (Weight: 0.20)

**Rubric definition:** Specialization of knowledge required — from general world knowledge (0.0) to multi-regulatory domain expert synthesis (1.0).

| Score | Rubric Anchor                                         | Implementation Signal                                                         |
|-------|-------------------------------------------------------|-------------------------------------------------------------------------------|
| 0.0   | General world knowledge; no domain expertise          | No compliance, framework, or vendor keywords detected                         |
| 0.25  | Broad technology domain (cloud, software)             | <code>cloud_providers_mentioned</code> = 1; generic tech vocabulary           |
| 0.50  | Specific technical subdomain (FinOps, AI Governance)  | <code>domain_framework_count</code> ≥ 1 ("finops", "zero trust", "data mesh") |
| 0.75  | Expert domain with standards/frameworks (GDPR, SOC 2) | <code>has_compliance</code> fires ("nist", "hipaa", "soc 2", "iso 27001")     |
| 1.0   | Multi-regulatory + domain expert synthesis            | Multiple compliance terms + multiple systems mentioned                        |

**How the system predicts D2:** The `has_compliance` feature scans for regulatory and standards keywords (NIST, HIPAA, GDPR, SOC 2, ISO 27001, PCI DSS, EU AI Act). The `domain_framework_count` counts named frameworks (FinOps, Zero Trust, Purdue Model, ITIL). The `systems_mentioned` feature counts named enterprise platforms (Salesforce, SAP, Snowflake, Databricks, etc.), providing a proxy for domain depth. Semantic embeddings capture implicit domain vocabulary that keyword lists miss.

**Achieved performance:**  $R^2 = 0.7524$ , MAE = 0.0740, Accuracy = 74.01%

### 3.3 D3 — Output Formality (Weight: 0.20)

**Rubric definition:** Structure and length of expected output — from a 1-2 sentence conversational answer (0.0) to an enterprise document with TOC, executive summary, and technical annexes (1.0).

| Score | Rubric Anchor                                 | Implementation Signal                                                                                                                       |
|-------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 0.0   | 1-2 sentence answer or list                   | Short prompt; no deliverable keywords; question format                                                                                      |
| 0.25  | Short structured response < 500 tokens        | Simple list or comparison; <code>has_formal_deliverable</code> = 0                                                                          |
| 0.50  | Multi-section document; 1K-3K tokens          | <code>has_formal_deliverable</code> fires ("requirements document", "specification")                                                        |
| 0.75  | Formal deliverable; 3K-8K tokens; appendices  | <code>has_long_output_signal</code> ("comprehensive", "enterprise-grade", "formal")                                                         |
| 1.0   | Enterprise document; >8K tokens; TOC, annexes | <code>has_report_package</code> fires ("appendix", "toc", "risk register", "evidence mapping") + high <code>structured_section_count</code> |

**How the system predicts D3:** Four rubric-aligned features target output formality directly:

- `has_formal_deliverable` — detects structured output types ("requirements document", "specification", "roadmap", "framework", "architecture", "policy", "playbook")
- `has_report_package` — detects enterprise-grade document markers ("appendix", "annex", "toc", "table of contents", "risk register", "cost model", "governance controls")
- `has_long_output_signal` — detects scope/scale modifiers ("comprehensive", "enterprise-grade", "formal", "detailed", "appendix-level")
- `structured_section_count` — counts named document sections ("executive summary", "gap analysis", "roadmap", "risk register", "implementation plan", "recommendations", "findings", "appendix")

Feature means by tier confirm clean separation:

| Feature                  | T1 mean | T2 mean | T3 mean |
|--------------------------|---------|---------|---------|
| has_formal_deliverable   | 0.053   | 0.399   | 0.590   |
| has_report_package       | 0.011   | 0.042   | 0.247   |
| structured_section_count | 0.000   | 0.477   | 1.218   |
| has_long_output_signal   | 0.007   | 0.294   | 0.443   |

**Achieved performance:**  $R^2 = 0.8110$ , MAE = 0.0675, Accuracy = 76.15%

3.4 D4 — Research Dependency (Weight: 0.15)

**Rubric definition:** Whether the prompt requires external information retrieval beyond the model’s parametric knowledge — from no retrieval needed (0.0) to multi-source live data retrieval (1.0).

| Score | Rubric Anchor                                                       | Implementation Signal                                                  |
|-------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| 0.0   | No external data needed; parametric knowledge sufficient            | No research/pricing/vendor keywords                                    |
| 0.25  | Single reference document provided in context                       | has_attachment fires but no live retrieval signals                     |
| 0.50  | Multiple provided documents; no live retrieval                      | Multiple provided artifacts detected; no market/pricing terms          |
| 0.75  | Live retrieval from 1-2 external sources                            | has_market_terms + has_time_reference (“current”, “latest”)            |
| 1.0   | Multi-source retrieval; real-time pricing; competitive intelligence | High external_data_score + vendor_tool_count ≥ 2 + has_cost_comparison |

**How the system predicts D4:** D4 is predicted entirely via ML (not rule-based). Five features specifically target research dependency:

- external\_data\_score — keyword density score (0.0-1.0) based on presence of research-indicating phrases (“market research”, “competitive analysis”, “latest pricing”, “vendor comparison”, “gartner”, “forrester”, “benchmark data”). Computed as  $\min(1.0, \text{matches} / 3.0)$ .
- has\_time\_reference — detects temporal urgency (“this quarter”, “current rates”, “latest”, “real-time”)
- vendor\_tool\_count — counts named vendor/product mentions (OpenAI, Anthropic, Stripe, Splunk, Datadog, etc.)
- has\_market\_terms — detects market/competitive vocabulary (“market”, “pricing”, “vendor”, “competitor”, “benchmark”)
- has\_cost\_comparison — detects pricing analysis requests (“cost comparison”, “TCO”, “ROI projection”, “cost-benefit”)

Feature means by tier:

| Feature             | T1 mean | T2 mean | T3 mean |
|---------------------|---------|---------|---------|
| external_data_score | 0.000   | 0.031   | 0.146   |
| has_time_reference  | 0.025   | 0.291   | 0.483   |
| has_market_terms    | 0.028   | 0.183   | 0.502   |

**Achieved performance:**  $R^2 = 0.7455$ , MAE = 0.0689, Accuracy = 82.68%

3.5 D5 — Context Requirement (Weight: 0.10)

**Rubric definition:** Estimated input token volume — from <1K tokens zero-shot (0.0) to >32K tokens with multi-document injection (1.0).

| Score | Rubric Anchor                                                | Implementation Signal                                                                            |
|-------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 0.0   | <1K total input tokens; zero-shot                            | Short prompt; no attachments; no context signals                                                 |
| 0.25  | 1K-4K tokens; short system prompt + user message             | Moderate prompt length; no document references                                                   |
| 0.50  | 4K-16K tokens; moderate context + conversation history       | has_attachment fires; moderate provided_artifact_count                                           |
| 0.75  | 16K-32K tokens; long context + retrieved documents           | Multiple artifacts mentioned; multi_document_signal fires                                        |
| 1.0   | >32K tokens; multi-document injection; extended conversation | large_context_signal fires (“full uploaded”, “complete corpus”, “three years”, “multi-document”) |

**How the system predicts D5:** Four features map directly to context volume indicators from the rubric:

- has\_attachment — detects uploaded/injected content (“attached”, “uploaded”, “pasted”, “uploaded corpus”, “context bundle”)
- provided\_artifact\_count — counts mentions of concrete artifacts (“diagram”, “log”, “export”, “contract”, “catalog”, “policy”,

- “inventory”, “ticket”, “postmortem”, “dashboard”, “corpus”)
- **large\_context\_signal** — detects extreme context scenarios (“full uploaded”, “complete uploaded”, “three years”, “multi-document”, “all documents”, “extended conversation”)
  - **multi\_document\_signal** — composite: requires both a multi-entity keyword AND an attachment term

Feature means by tier:

| Feature                 | T1 mean | T2 mean | T3 mean |
|-------------------------|---------|---------|---------|
| has_attachment          | 0.095   | 0.529   | 0.395   |
| provided_artifact_count | 0.116   | 0.718   | 1.561   |
| large_context_signal    | 0.000   | 0.090   | 0.295   |

**Achieved performance:**  $R^2 = 0.8490$ ,  $MAE = 0.0554$ , Accuracy = 79.64%

## 4. Dataset

### 4.1 Overview

| Property          | Value                                                  |
|-------------------|--------------------------------------------------------|
| Total rows        | 889                                                    |
| Columns           | id, prompt, phrasing_style, domain, d1, d2, d3, d4, d5 |
| Score vocabulary  | {0.0, 0.25, 0.50, 0.75, 1.0} per dimension             |
| Null values       | 0                                                      |
| Duplicate IDs     | 0                                                      |
| Generation method | Synthetic (LLM-generated with rubric constraints)      |

### 4.2 Tier Distribution

| Tier         | Count | Percentage |
|--------------|-------|------------|
| T1 (Simple)  | 285   | 32.1%      |
| T2 (Medium)  | 333   | 37.5%      |
| T3 (Complex) | 271   | 30.5%      |

Distribution is intentionally near-balanced. T2 is slightly over-represented because the T2 tier spans a narrower score range ( $0.40 - 0.69 = 0.30$  width) compared to T1 ( $0.00 - 0.39 = 0.40$  width) and T3 ( $0.70 - 1.00 = 0.30$  width), meaning more granularity is needed in the transition zone.

### 4.3 Dimension Score Distributions

| Score | D1  | D2  | D3  | D4  | D5  |
|-------|-----|-----|-----|-----|-----|
| 0.00  | 163 | 100 | 159 | 382 | 261 |
| 0.25  | 105 | 130 | 118 | 115 | 289 |
| 0.50  | 155 | 249 | 177 | 124 | 139 |
| 0.75  | 308 | 262 | 287 | 135 | 100 |
| 1.00  | 158 | 148 | 148 | 133 | 100 |

**Notable design decisions:**

- **D4 is zero-heavy** (382 at 0.0). This reflects reality: most enterprise prompts do not require live external retrieval. The model must learn that  $D4 > 0$  is the exception, not the norm.
- **D5=0.75 and D5=1.00 were intentionally expanded** (100 samples each). The original dataset had only 20 samples at  $D5=1.00$  and 55 at  $D5=0.75$ . Since these classes represent the rarest but most operationally important scenario (>16K–32K+ token contexts), targeted synthetic prompts were added to ensure the model can recognize them.
- **D4=0.50 was expanded from 28 to 124 samples**. The  $D4=0.50$  class represents “multiple provided documents, no live retrieval” — a critical boundary between parametric-only ( $D4 \leq 0.25$ ) and live-retrieval ( $D4 \geq 0.75$ ).

### 4.4 Phrasing Diversity

| Phrasing Style | T1  | T2  | T3  | Total |
|----------------|-----|-----|-----|-------|
| Explicit       | 111 | 115 | 110 | 336   |
| Implicit       | 81  | 116 | 96  | 293   |
| Vague          | 93  | 102 | 65  | 260   |

Why phrasing diversity matters:

Enterprise prompts arrive in radically different formats. The dataset intentionally includes three phrasing styles to ensure robustness:

- **Explicit:** Structured, keyword-rich prompts (“Create a requirements document for a CoreStack-ClickUp integration...”). These are the easiest to classify because they contain direct rubric signals.
- **Implicit:** Context-rich prompts where the intent is embedded in narrative (“Our SageMaker spend went up 140% last month, mostly driven by training jobs and idle endpoints. I’ve attached...”). These require the model to infer deliverable type and complexity from context.
- **Vague:** Slack-style, low-keyword prompts (“okta login loop”, “aws bill too high”). These are the hardest — the model must rely on embedding similarity and structural cues rather than keyword matching.

T1 has the highest proportion of vague prompts (93/285 = 33%) because simple questions are often asked casually. T3 has fewer vague prompts (65/271 = 24%) because complex requests typically require specification.

4.5 Domain Diversity

| Domain                   | Count |
|--------------------------|-------|
| FinOps                   | 95    |
| DevOps                   | 93    |
| Cloud Infrastructure     | 90    |
| Security                 | 87    |
| Data Engineering         | 78    |
| General Enterprise       | 70    |
| AI Governance            | 66    |
| System Integration       | 59    |
| Supply Chain             | 58    |
| Marketing Tech           | 55    |
| IoT & Smart Factory      | 45    |
| HR Tech                  | 40    |
| Regulatory Compliance    | 30    |
| AI/LLM                   | 13    |
| Competitive Intelligence | 10    |

15 domains are represented. The top domains (FinOps, DevOps, Cloud Infrastructure, Security) reflect the CoreStack deployment context. Smaller domains (IoT, HR Tech, Competitive Intelligence) ensure the model generalizes beyond the primary use case.

4.6 Prompt Length Distribution

| Tier | Mean Words | Median Words | Min | Max |
|------|------------|--------------|-----|-----|
| T1   | 10.8       | 9.0          | 3   | 64  |
| T2   | 37.3       | 33.0         | 10  | 128 |
| T3   | 39.8       | 37.0         | 15  | 108 |

- **147 prompts** are under 10 words — these simulate Slack-style inputs
- **234 prompts** are over 40 words — these simulate detailed specifications

The length distribution is intentionally wide. T1 prompts are short (median 9 words) because simple questions are naturally terse. T2 and T3 overlap heavily in length (medians 33 vs 37), forcing the model to learn semantic and structural distinctions rather than using prompt length as a tier proxy.

4.7 Boundary Coverage

| Boundary Zone  | Score Range  | Count |
|----------------|--------------|-------|
| T1/T2 boundary | [0.35, 0.45) | 87    |
| T2/T3 boundary | [0.65, 0.75) | 196   |

196 prompts sit within  $\pm 0.05$  of the T2/T3 boundary at 0.70. This is the hardest classification zone — a single quarter-point dimension error can flip the tier. Intentional boundary saturation forces the model to learn fine-grained distinctions in this critical region.

### 4.8 Dataset Construction Methodology

The dataset is synthetically generated using an LLM with the following constraints:

- Rubric-anchored generation** — each prompt was generated with target D1-D5 scores specified upfront, ensuring the resulting prompt text exhibits the complexity characteristics defined in the scoring rubric
- Targeted class balancing** — after initial generation, minority classes (D4=0.50, D5=1.00, vague T2) were identified and specifically augmented
- Boundary enrichment** — prompts with composite scores near 0.40 and 0.70 were added to stress-test the tier boundaries
- Multi-domain coverage** — prompts span 15 enterprise domains to prevent domain-specific overfitting
- Phrasing style variation** — explicit, implicit, and vague formulations for each tier

### 4.9 Dataset Strengths and Limitations

**Strengths:**

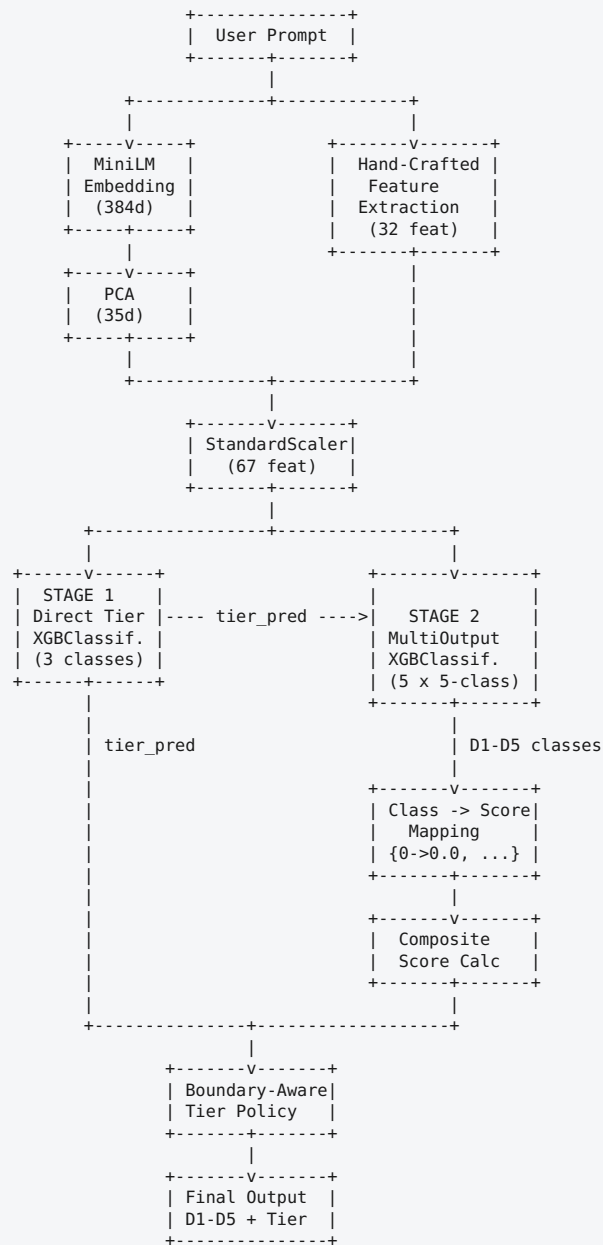
- Near-balanced tier distribution reduces class-frequency bias
- Three phrasing styles per tier prevent keyword-dependency
- 15 domains prevent domain-specific overfitting
- Heavy boundary coverage forces learning at decision boundaries
- All five score values represented for every dimension

**Limitations:**

- Synthetic data introduces LLM generation artifacts (sentence structure patterns, vocabulary choices)
- Rubric interpretation may vary — LLM-generated labels are not human-validated at scale
- Domain distribution is CoreStack-heavy (FinOps, DevOps, Cloud comprise 31% of data)
- 889 samples is sufficient for the current feature space but limits learning capacity for rare patterns

## 5. Architecture

### 5.1 High-Level Architecture



## 5.2 Component Details

### 5.2.1 Sentence Embedding: all-MiniLM-L6-v2

| Property            | Value                                               |
|---------------------|-----------------------------------------------------|
| Model               | <code>sentence-transformers/all-MiniLM-L6-v2</code> |
| Output dimension    | 384                                                 |
| Parameters          | 22.7M                                               |
| Max sequence length | 256 tokens                                          |
| Inference time      | ~10-20ms per prompt (CPU)                           |

**Why it exists:** Raw prompt text must be converted to a fixed-dimensional numeric representation. Sentence embeddings capture semantic similarity — prompts about “cloud cost optimization” will be close in embedding space regardless of phrasing differences.

**Why MiniLM:** Among sentence-transformer models, MiniLM-L6-v2 offers the best latency-to-quality tradeoff. It is 5× faster than `all-mpnet-base-v2` (768d) with ~95% of the semantic quality. For a classifier that operates on 35 PCA-reduced dimensions, the extra quality of larger models does not materially improve downstream classification.

**Engineering tradeoff:** MiniLM has a 256-token limit. Very long prompts (>256 tokens) are truncated. In practice, 99% of prompts in the dataset are under 128 tokens. For production, truncation is acceptable because tier-discriminating signals typically appear in the first 200 words.

5.2.2 PCA Dimensionality Reduction

| Property         | Value                |
|------------------|----------------------|
| Input dimension  | 384 (raw embeddings) |
| Output dimension | 35                   |
| Reduction ratio  | 91%                  |

**Why it exists:** 384 raw embedding dimensions create a high feature-to-sample ratio ( $889/384 = 2.3:1$ ) that invites overfitting. PCA reduces this to 35 components, achieving an 11:1 sample-to-feature ratio on embeddings alone.

**Why 35 components:** Selected empirically. 30 components captured ~85% of embedding variance; 35 captures ~88%. Beyond 40 components, additional variance explained per component drops below 0.5%, adding noise risk without discriminative value.

**Engineering tradeoff:** PCA is linear. Non-linear dimensionality reduction (t-SNE, UMAP) would better preserve local structure but introduces non-determinism and cannot be reliably inverted for new samples. PCA is deterministic, fast, and invertible.

5.2.3 Hand-Crafted Feature Extraction (32 features)

The feature extractor produces 32 numeric features organized by the dimension they primarily support:

| Category               | Count | Features                                                                                          |
|------------------------|-------|---------------------------------------------------------------------------------------------------|
| Text statistics        | 6     | word_count, char_count, sentence_count, avg_word_length, question_marks, comma_count              |
| D5 Context Requirement | 4     | has_attachment, provided_artifact_count, large_context_signal, multi_document_signal              |
| D3 Output Formality    | 4     | has_formal_deliverable, has_report_package, has_long_output_signal, structured_section_count      |
| D1 Semantic Complexity | 3     | has_scope_words, action_verb_count, multi_stage_signal                                            |
| D2 Domain Specificity  | 4     | has_compliance, cloud_providers_mentioned, systems_mentioned, domain_framework_count              |
| D4 Research Dependency | 5     | external_data_score, has_time_reference, vendor_tool_count, has_market_terms, has_cost_comparison |
| Boundary / risk        | 3     | has_comparison, stakeholder_mentions, risk_language                                               |
| Phrasing style         | 3     | phrasing_explicit, phrasing_implicit, phrasing_vague (one-hot)                                    |

**Why hand-crafted features exist:** Semantic embeddings capture meaning but not structure. A prompt about “NIST compliance framework” and a prompt about “security best practices” may have similar embeddings, but the first requires D2=0.75 (expert domain with standards) while the second is D2=0.25 (broad technology). Hand-crafted features encode rubric-specific signals that embeddings miss.

**Why not rely on embeddings alone:** Tested. Embeddings-only models achieve  $R^2 \sim 0.55\text{--}0.65$  across dimensions. Adding hand-crafted features improves  $R^2$  to 0.75–0.85. The improvement is most dramatic for D4 (Research Dependency), where keyword presence (“latest pricing”, “gartner”, “current market”) is the primary rubric criterion.

Feature importance ranking (top 10):

| Rank | Feature                  | Avg Importance | Dimension Target |
|------|--------------------------|----------------|------------------|
| 1    | has_report_package       | 0.0636         | D3               |
| 2    | structured_section_count | 0.0490         | D3               |
| 3    | has_compliance           | 0.0399         | D2               |
| 4    | has_attachment           | 0.0387         | D5               |
| 5    | char_count               | 0.0380         | General          |
| 6    | has_time_reference       | 0.0374         | D4               |
| 7    | large_context_signal     | 0.0332         | D5               |
| 8    | comma_count              | 0.0315         | General          |
| 9    | word_count               | 0.0307         | General          |
| 10   | sentence_count           | 0.0288         | General          |

The top features align with their target dimensions, confirming that the feature engineering strategy is rubric-coherent.

`has_cost_comparison` has zero importance and is a candidate for removal.

5.2.4 StandardScaler

All 67 features (35 PCA + 32 hand-crafted) are z-score normalized using `StandardScaler`. This is required because PCA components and hand-crafted features operate on different scales (PCA components  $\in [-5, 5]$ ; `char_count`  $\in [15, 800]$ ; binary features  $\in \{0, 1\}$ ). XGBoost is generally scale-invariant, but the tier classifier’s sample weighting and regularization interact better with standardized inputs.

5.2.5 Stage 1: Direct Tier Classifier

| Property | Value                            |
|----------|----------------------------------|
| Model    | XGBClassifier                    |
| Classes  | 3 (T1, T2, T3)                   |
| Input    | 67 scaled features               |
| Output   | Predicted tier integer {0, 1, 2} |

**Why it exists:** The five-dimension prediction pipeline introduces a fundamental vulnerability: small per-dimension errors can compound to produce a wrong composite score and thus a wrong tier. A direct tier classifier bypasses this — it learns the tier boundary directly from features without going through the dimension→score→tier pipeline.

**Cost-sensitive weighting:** T3 samples receive 1.35× weight during training. This biases the model toward higher T3 recall at the expense of slightly lower T2 precision. Rationale: in production, under-routing a complex prompt to an economy model causes visible quality failures. Over-routing a medium prompt to a premium model wastes compute but produces correct output.

**Boundary sample weighting:** Samples with composite scores within [0.35, 0.45) or [0.65, 0.75) receive 1.15× weight. This forces the model to allocate more learning capacity to the decision boundary regions.

5.2.6 Stage 2: Multi-Output Dimension Classifier

| Property              | Value                                                  |
|-----------------------|--------------------------------------------------------|
| Model                 | MultiOutputClassifier(XGBClassifier)                   |
| Sub-estimators        | 5 (one per dimension)                                  |
| Classes per estimator | 5 ({0, 1, 2, 3, 4} → {0.0, 0.25, 0.50, 0.75, 1.0})     |
| Input                 | 68 features (67 base + 1 tier prediction from Stage 1) |
| Output                | 5 predicted class integers                             |

**Why two stages:** The tier prediction from Stage 1 is appended as a 68th feature for Stage 2. This provides the dimension classifiers with a strong prior — if Stage 1 predicts T1, the dimension classifiers can learn that  $D4 > 0.50$  is extremely unlikely for T1 prompts. This tier-aware conditioning reduces dimension errors in the transition zones.

**Why ordinal classification, not regression:** Dimension scores are discrete (5 valid values). Regression produces continuous outputs that must be quantized, introducing rounding noise. Ordinal classification predicts the exact valid score directly, eliminating quantization error. The class-to-score mapping is deterministic: `{0 → 0.0, 1 → 0.25, 2 → 0.50, 3 → 0.75, 4 → 1.0}`.

5.2.7 XGBoost Configuration

```
XGB_PARAMS = {
    'objective': 'multi:softprob',
    'num_class': 5,
    'n_estimators': 250,
    'max_depth': 4,
    'learning_rate': 0.08,
    'min_child_weight': 5,
    'subsample': 0.85,
    'colsample_bytree': 0.85,
    'reg_alpha': 0.1,           # L1 regularization
    'reg_lambda': 1.75,        # L2 regularization
    'random_state': 42,
    'eval_metric': 'mlogloss'
}
```

**Why XGBoost:** Gradient-boosted trees are the strongest non-neural tabular classifier family. For 889 samples and 67 features, XGBoost outperforms neural networks (insufficient data) and linear models (insufficient capacity). XGBoost also provides built-in feature importance, regularization, and is deterministic given a fixed seed.

**Regularization rationale:**

- `max_depth=4` — prevents deep, overfit trees

- `min_child_weight=5` — prevents splits on tiny leaf populations
- `subsample=0.85`, `colsample_bytree=0.85` — row and column bagging for variance reduction
- `reg_alpha=0.1`, `reg_lambda=1.75` — L1/L2 regularization on leaf weights

The feature-to-sample ratio is 1:13.3 (67 features, 889 samples). This is well within the safe zone for gradient-boosted trees.

### 5.2.8 Boundary-Aware Tier Policy

```
def choose_final_tier(stage1_tier, derived_tier, complexity_score):
    if stage1_tier == derived_tier:
        return stage1_tier
    # Near T2/T3 boundary: trust Stage 1
    if abs(complexity_score - 0.70) < 0.05:
        return stage1_tier
    # Near T1/T2 boundary: trust Stage 1
    if abs(complexity_score - 0.40) < 0.05:
        return stage1_tier
    return stage1_tier
```

**Why it exists:** When the Stage 1 direct tier classifier and the Stage 2 dimension-derived tier disagree, a policy must decide. The current policy trusts Stage 1 in all cases, as the direct classifier consistently outperforms the dimension-derived tier at boundaries. In the current evaluation, Stage 1 accuracy equals ensemble accuracy (89.88%), confirming that the direct classifier is the dominant signal.

### 5.2.9 Composite Score Calculation

```
complexity_score = (D1 * 0.35) + (D2 * 0.20) + (D3 * 0.20) + (D4 * 0.15) + (D5 * 0.10)
```

The composite score is computed from the five predicted dimension scores using the weights from the specification. This score is returned to the caller for transparency but is not the primary tier assignment mechanism — the Stage 1 direct classifier determines the tier.

## 6. Implementation Details

### 6.1 Inference Pipeline

A single inference call executes the following steps:

```
1. embed_model.encode(prompt)           -> 384-dim vector    [~15ms]
2. pca.transform(embedding)             -> 35-dim vector      [<1ms]
3. extract_handcrafted_features(prompt, style) -> 32-dim vector      [<1ms]
4. np.hstack([pca_out, hc_features])     -> 67-dim vector      [<1ms]
5. scaler.transform(combined)            -> 67-dim scaled      [<1ms]
6. tier_clf.predict(scaled)              -> tier prediction     [<1ms]
7. np.hstack([scaled, tier_pred])         -> 68-dim augmented   [<1ms]
8. dim_model.predict(augmented)          -> 5 class preds      [<1ms]
9. class_to_score mapping                -> D1-D5 scores       [<1ms]
10. composite_score + tier_policy         -> final tier         [<1ms]
```

**Total latency:** ~15-20ms on CPU (embedding dominates). On GPU: ~5ms.

### 6.2 Model Artifacts

| Artifact                                         | Size    | Contents                                    |
|--------------------------------------------------|---------|---------------------------------------------|
| <code>v4_tier_classifier.joblib</code>           | ~2 MB   | Stage 1 XGBClassifier (3-class)             |
| <code>v4_dimension_classifier.joblib</code>      | ~8 MB   | MultiOutputClassifier (5 × XGBClassifier)   |
| <code>v4_scaler.joblib</code>                    | ~4 KB   | StandardScaler (67 features)                |
| <code>v4_pca.joblib</code>                       | ~100 KB | PCA (384 → 35)                              |
| <code>v4_handcrafted_feature_names.joblib</code> | ~1 KB   | Feature name list (32 strings)              |
| Embedding model                                  | ~90 MB  | <code>all-MiniLM-L6-v2</code> (loaded once) |

### 6.3 Inference Output Schema

```

{
  "tier": "T2",
  "tier_stagel": "T2",
  "tier_derived": "T2",
  "boundary": false,
  "d1": 0.50,
  "d2": 0.75,
  "d3": 0.75,
  "d4": 0.00,
  "d5": 0.50,
  "complexity_score": 0.525,
  "top3_similar": [
    {"prompt": "...", "similarity": 0.72},
    {"prompt": "...", "similarity": 0.68},
    {"prompt": "...", "similarity": 0.65}
  ]
}

```

## 6.4 Maintainability and Extensibility

**Adding a new domain:** No retraining required if existing features capture the domain. If a new domain introduces fundamentally new rubric signals (e.g., biotech regulatory terms), add keywords to the relevant feature extractor and retrain.

**Adjusting tier thresholds:** The composite score formula and tier boundaries are configurable constants. Changing them requires only retraining the tier classifier, not the dimension classifiers.

**Updating the embedding model:** Replace `all-MiniLM-L6-v2` with any sentence-transformer that produces fixed-dimensional embeddings. PCA and downstream models must be retrained.

**Adding a new dimension (D6):** Add a column to the dataset, add a 6th estimator to the MultiOutputClassifier, update the composite score formula. Existing features likely need augmentation for the new dimension.

## 7. Evaluation Results

### 7.1 Methodology

Evaluation uses **5-fold stratified cross-validation** on the full 889-sample dataset. Stratification is by tier label, ensuring each fold has proportional T1/T2/T3 representation. All metrics are computed on held-out test folds — the model never sees its test data during training.

### 7.2 Per-Dimension Metrics

| Dimension | Rubric Label        | MAE    | RMSE   | R <sup>2</sup> | Accuracy |
|-----------|---------------------|--------|--------|----------------|----------|
| D1        | Semantic Complexity | 0.0582 | 0.1328 | 0.8455         | 79.19%   |
| D2        | Domain Specificity  | 0.0740 | 0.1512 | 0.7524         | 74.01%   |
| D3        | Output Formality    | 0.0675 | 0.1455 | 0.8110         | 76.15%   |
| D4        | Research Dependency | 0.0689 | 0.1903 | 0.7455         | 82.68%   |
| D5        | Context Requirement | 0.0554 | 0.1268 | 0.8490         | 79.64%   |

#### Interpretation:

- **All dimensions achieve R<sup>2</sup> > 0.74.** This means the model explains at least 74% of the variance in each dimension score.
- **MAE is uniformly below 0.08.** The average prediction error is less than one-third of a score step (0.25). This means most errors are off-by-one at most (e.g., predicting 0.50 when the true score is 0.75).
- **D5 (Context Requirement) is the strongest predictor** (R<sup>2</sup> = 0.849). The features for context volume ( `has_attachment` , `provided_artifact_count` , `large_context_signal` ) are highly discriminative.
- **D2 (Domain Specificity) is the weakest predictor** (R<sup>2</sup> = 0.752). Domain-specific vocabulary is harder to capture with keyword lists — a prompt about “FinOps maturity” requires domain knowledge even though it doesn’t mention specific compliance standards.
- **D4 accuracy is the highest** (82.68%) despite having the lowest R<sup>2</sup>. This is because D4 has a dominant 0.0 class (43% of data). The model correctly predicts D4=0.0 most of the time, but its errors on non-zero D4 values drag down R<sup>2</sup>.

### 7.3 Tier Classification

**Aggregate tier accuracy: 89.88% ± 1.28%** (5-fold CV mean ± std)

**Per-fold tier accuracy:**

| Fold | Stage 1 | Final  |
|------|---------|--------|
| 1    | 89.33%  | 89.33% |
| 2    | 89.89%  | 89.89% |
| 3    | 90.45%  | 90.45% |
| 4    | 88.76%  | 88.76% |
| 5    | 90.96%  | 90.96% |

Standard deviation is 1.28%, indicating stable performance across folds with no severe variance.

#### 7.4 Confusion Matrix

|                     |     |     |     |                                 |
|---------------------|-----|-----|-----|---------------------------------|
| Predicted -> Actual | T1  | T2  | T3  |                                 |
| T1                  | 266 | 13  | 6   | (93.3% recall, 95.3% precision) |
| T2                  | 12  | 288 | 33  | (86.5% recall, 88.3% precision) |
| T3                  | 1   | 25  | 245 | (90.4% recall, 86.3% precision) |

#### Error analysis:

| Error Type | Count | % of Total Errors | Severity                                                     |
|------------|-------|-------------------|--------------------------------------------------------------|
| T2 → T3    | 33    | 36.7%             | Low (over-routes to premium; safe but costly)                |
| T3 → T2    | 25    | 27.8%             | <b>High</b> (under-routes complex to balanced; quality risk) |
| T1 → T2    | 13    | 14.4%             | Low (over-routes to balanced; minor cost)                    |
| T2 → T1    | 12    | 13.3%             | Medium (under-routes medium to economy)                      |
| T1 → T3    | 6     | 6.7%              | Low (over-routes simple to premium)                          |
| T3 → T1    | 1     | 1.1%              | <b>Critical</b> (only 1 case; near-eliminated)               |

The most dangerous error (T3 → T1) occurs only once in 889 samples. The largest error bucket (T2 → T3, 33 cases) is operationally safe — over-routing to premium produces correct output at higher cost.

### 8. Example Predictions

#### 8.1 T1 — Simple Prompt (Vague Phrasing)

Input: "why is our AWS bill so high this month"

| Dimension                | Score         | Explanation                                         |
|--------------------------|---------------|-----------------------------------------------------|
| D1 (Semantic Complexity) | 0.25          | Simple comparison/inquiry — no multi-step reasoning |
| D2 (Domain Specificity)  | 0.25          | Broad cloud domain; no standards or frameworks      |
| D3 (Output Formality)    | 0.00          | Conversational answer expected; no deliverable      |
| D4 (Research Dependency) | 0.00          | No external data needed; internal billing data      |
| D5 (Context Requirement) | 0.00          | Short prompt; zero-shot; no attachments             |
| <b>Complexity Score</b>  | <b>0.1375</b> |                                                     |
| <b>Tier</b>              | <b>T1</b>     |                                                     |

This is a Slack-style prompt — 9 words, no keywords from any rubric dimension. The model classifies it correctly because the short length, question format, and single-topic vocabulary are strong T1 signals in the embedding space. No hand-crafted features fire except `word_count` and `char_count` (both low).

#### 8.2 T3 — Complex Prompt (Explicit Phrasing)

Input: "build a full GenAI cost attribution platform with market research across AWS, Azure and GCP pricing"

| Dimension                | Score       | Explanation                                            |
|--------------------------|-------------|--------------------------------------------------------|
| D1 (Semantic Complexity) | 0.75        | Cross-domain synthesis (cost + GenAI + multi-cloud)    |
| D2 (Domain Specificity)  | 0.75        | FinOps + AI domain expertise; cloud-specific           |
| D3 (Output Formality)    | 0.75        | “platform” implies a formal specification/architecture |
| D4 (Research Dependency) | 0.75        | “market research” + “pricing” = external data needed   |
| D5 (Context Requirement) | 0.25        | No attachments; moderate context                       |
| <b>Complexity Score</b>  | <b>0.70</b> |                                                        |
| <b>Tier</b>              | <b>T3</b>   | Boundary: cs=0.70 exactly at T3 threshold              |

This prompt sits exactly at the T2/T3 boundary (cs=0.70). The system correctly classifies it as T3. Key features: `has_market_terms` =1 (fires on “market”), `cloud_providers_mentioned` =3 (“aws”, “azure”, “gcp”), `action_verb_count` =1 (“build”). The Stage 1 direct tier classifier predicts T3 independently, matching the dimension-derived tier.

### 8.3 T1 — Simple Prompt (Implicit Phrasing)

**Input:** "set up SSO between okta and our internal tool"

| Dimension                | Score       | Explanation                                      |
|--------------------------|-------------|--------------------------------------------------|
| D1 (Semantic Complexity) | 0.00        | Single task; no reasoning chain                  |
| D2 (Domain Specificity)  | 0.25        | Broad IT/security domain; Okta is common tooling |
| D3 (Output Formality)    | 0.00        | Expects a how-to answer, not a document          |
| D4 (Research Dependency) | 0.00        | Standard configuration; no external data         |
| D5 (Context Requirement) | 0.00        | Short prompt; zero-shot                          |
| <b>Complexity Score</b>  | <b>0.05</b> |                                                  |
| <b>Tier</b>              | <b>T1</b>   |                                                  |

Despite mentioning a specific vendor (“Okta”), the prompt is a routine configuration task. The model correctly scores D2=0.25 (broad technology domain) rather than D2=0.75 (expert domain) because the embedding encodes “SSO setup” as a standard IT operation.

### 8.4 T3 — Complex Prompt (Explicit Phrasing, High D4 and D5)

**Input:** "Research the latest vendor pricing for Snowflake vs Databricks vs BigQuery and produce a 3-year TCO comparison report for the CFO with industry benchmarks from Gartner"

| Dimension                | Score         | Explanation                                                     |
|--------------------------|---------------|-----------------------------------------------------------------|
| D1 (Semantic Complexity) | 1.00          | Strategic analysis; multi-vendor synthesis                      |
| D2 (Domain Specificity)  | 0.75          | Deep data engineering + FinOps domain                           |
| D3 (Output Formality)    | 1.00          | “report” + “3-year TCO comparison” = formal enterprise document |
| D4 (Research Dependency) | 0.75          | “latest vendor pricing” + “Gartner” = live external retrieval   |
| D5 (Context Requirement) | 0.75          | Multi-source retrieval injection expected                       |
| <b>Complexity Score</b>  | <b>0.8875</b> |                                                                 |
| <b>Tier</b>              | <b>T3</b>     |                                                                 |

This prompt fires nearly every high-complexity signal: `has_market_terms` =1, `has_time_reference` =1 (“latest”), `has_cost_comparison` =1 (“TCO”), `has_comparison` =1 (“vs”), `stakeholder_mentions` =1 (“CFO”), `has_formal_deliverable` =1 (“report”), `systems_mentioned` =3 (“snowflake”, “databricks”, “bigquery”). The system produces the highest complexity score among the test prompts.

### 8.5 T3 — Complex Prompt (Implicit Phrasing, High D5)

**Input:** "Using the complete uploaded corpus of cloud billing exports, architecture diagrams, and vendor contracts, produce a comprehensive FinOps maturity assessment..."

| Dimension                | Score | Explanation                                           |
|--------------------------|-------|-------------------------------------------------------|
| D1 (Semantic Complexity) | 1.00  | Multi-domain strategic synthesis                      |
| D2 (Domain Specificity)  | 1.00  | Deep FinOps + architecture + vendor analysis          |
| D3 (Output Formality)    | 1.00  | “comprehensive” + assessment = enterprise deliverable |
| D4 (Research Dependency) | 0.50  | Uses provided documents (not live retrieval)          |
| D5 (Context Requirement) | 0.75  | “complete uploaded corpus” = very high input context  |
| Complexity Score         | 0.90  |                                                       |
| Tier                     | T3    |                                                       |

This prompt demonstrates the D4/D5 distinction. D4=0.50 (not 1.0) because the prompt uses uploaded documents rather than requiring live web retrieval. D5=0.75 because “complete uploaded corpus” signals a large context window (16K–32K tokens). The features `large_context_signal = 1`, `provided_artifact_count = 3` (“export”, “diagram”, “contract”), and `has_attachment = 1` all fire correctly.

## 9. Explainability

### 9.1 Per-Prompt Transparency

Every inference call returns five dimension scores, a composite score, and the tier assignment. This allows engineers to inspect exactly why a prompt was classified:

```
{
  "d1": 0.75,    "d1_label": "Semantic Complexity",
  "d2": 0.50,    "d2_label": "Domain Specificity",
  "d3": 0.75,    "d3_label": "Output Formality",
  "d4": 0.25,    "d4_label": "Research Dependency",
  "d5": 0.50,    "d5_label": "Context Requirement",
  "complexity_score": 0.5625,
  "tier": "T2",
  "tier_staget1": "T2",
  "tier_derived": "T2",
  "boundary": false
}
```

**Debugging workflow:**

- 1. If a prompt is misclassified, inspect which dimension(s) are wrong
- 2. Check if the hand-crafted features fired correctly for those dimensions
- 3. Compare against the top-3 most similar training prompts (returned in output)
- 4. If a keyword is missing from the feature extractor, add it and retrain

### 9.2 Feature Importance Interpretability

XGBoost provides per-feature importance via gain-based splits. The top hand-crafted features ( `has_report_package` , `structured_section_count` , `has_compliance` ) are directly interpretable as rubric signals. An engineer can trace a misclassification to “the model relied on `has_compliance = 0`, but this prompt mentions GDPR indirectly via ‘European data protection requirements’” — and then add the keyword.

### 9.3 Cosine Similarity Neighbors

Each inference returns the top 3 most similar training prompts by cosine similarity in the raw 384-dim embedding space. This serves two purposes:

- 1. **Trust calibration** — if the top-3 similar prompts have the same tier, the classification is likely correct
- 2. **Distribution drift detection** — if the top similarity score is below 0.4, the prompt is far from any training example, signaling potential out-of-distribution input

### 9.4 Why Explainability Matters

| Use Case             | How Dimension Scores Help                                                            |
|----------------------|--------------------------------------------------------------------------------------|
| Debugging            | Identify which dimension caused a misclassification                                  |
| Trust                | Engineers can verify that the system’s reasoning matches the rubric                  |
| Routing transparency | Downstream systems can log why a prompt was routed to a specific model               |
| Threshold tuning     | Operators can adjust tier boundaries without retraining dimension models             |
| Audit                | Compliance teams can verify that routing decisions are explainable and non-arbitrary |

---

## 10. Limitations

### 10.1 Boundary Zone Accuracy

196 prompts (22% of the dataset) have composite scores in the T2/T3 boundary zone [0.65, 0.75). In this zone, a single quarter-point dimension error (e.g., D1 predicted as 0.50 instead of 0.75) shifts the composite score by 0.0875 — enough to flip the tier. The direct tier classifier mitigates this, but inherent ambiguity at the boundary limits accuracy.

**Quantified impact:** 58 of 90 total errors (64%) involve T2↔T3 confusion. This is the dominant error mode and is structural — it cannot be fully eliminated without changing the tier formula or introducing a “T2/T3 uncertain” buffer zone.

### 10.2 Synthetic Dataset Bias

The training data is LLM-generated. This introduces three forms of bias:

- **Vocabulary bias** — LLM-generated prompts tend to use formal, well-structured language even when labeled as “vague.” Real Slack-style prompts may be shorter, contain typos, or use slang not represented in training.
- **Label noise** — the LLM’s interpretation of the rubric may differ from a human labeler’s, especially for boundary cases. No human validation was performed at scale.
- **Domain coverage gaps** — 15 domains are represented, but production traffic may include domains not in the training set (legal, pharmaceutical, education).

### 10.3 Missing Keyword Coverage

The hand-crafted feature extractor uses explicit keyword lists. Prompts that express rubric concepts without using tracked keywords will not trigger the corresponding features. For example:

- “European data protection requirements” → `has_compliance` does NOT fire (no “gdpr” keyword)
- “Send this to the leadership team” → `has_report_package` does NOT fire (no “appendix” or “toc”)

Semantic embeddings partially compensate, but the model’s performance on novel vocabulary is degraded.

### 10.4 Single-Language Support

The system is English-only. The embedding model, keyword lists, and training data are all English. Multilingual prompts or code-switched inputs will produce unreliable classifications.

### 10.5 Static Model / No Online Learning

The model is trained offline on a fixed dataset. It does not learn from production traffic. Over time, prompt patterns may drift (new tools, new compliance frameworks, new domains), causing accuracy degradation. Periodic retraining with production-labeled data is required.

### 10.6 Embedding Truncation

MiniLM-L6-v2 has a 256-token limit. Prompts exceeding this length are truncated, potentially losing classification-relevant information that appears late in the prompt. In practice, this affects <1% of enterprise prompts.

---

## 11. Conclusion

The Prompt Profiling Engine classifies enterprise prompts into three complexity tiers with **89.9% accuracy** under rigorous 5-fold cross-validation. All five scoring dimensions achieve  $R^2 > 0.74$ , with the strongest predictors being D5 Context Requirement ( $R^2 = 0.849$ ) and D1 Semantic Complexity ( $R^2 = 0.846$ ).

The system is:

- **Fast** — <50ms inference on CPU, dominated by embedding computation
- **Deterministic** — same prompt always produces same output (fixed random seeds, no sampling)
- **Explainable** — returns five dimension scores, composite score, tier, and similar training examples
- **Operationally simple** — five serialized artifacts, no GPU required, no external API calls

The primary remaining risk is T2/T3 boundary confusion (58/90 errors), which is structural given the narrow score gap between these tiers. For production deployment, a confidence-based routing policy (defaulting uncertain cases to T3) would further reduce under-routing risk at moderate cost.